JAVA

Competitive Programming — Complete Reference

Helper methods for Arrays · Strings · HashMap · Stack · Queue · PriorityQueue · Bit tricks — and everything in between

Prepared for TCS NQT & general competitive-programming use

Table of Contents

Table of Contents

15 sections • Ctrl+click any row to jump straight there

*T.C = Time Complexity, S.C = Space Complexity. * = amortized / average case — worst case can differ (e.g. hash collisions, dynamic array resizing).*

1. Arrays (java.util.Arrays)

Method	T.C	S.C	Use
Arrays.sort(arr)	*O(n log n)*	*O(log n)*	Sorts the array ascending, in place. Example: [3,1,2] → [1,2,3]. O(n log n).
Arrays.sort(arr, from, to)	*O(n log n)*	*O(log n)*	Sorts only the sub-range [from, to) (to is exclusive), leaving the rest untouched.
Arrays.sort(arr, cmp)	*O(n log n)*	*O(n)*	Sorts using a custom Comparator. Only works on Object[] (e.g. Integer[]) or 2D arrays — NOT plain int[]/double[], since primitives have no Comparator overload.
Arrays.fill(arr, val)	*O(n)*	*O(1)*	Overwrites every element with val. Example: Arrays.fill(arr, -1) → all -1s.
Arrays.fill(arr, from, to, val)	*O(n)*	*O(1)*	Overwrites only indices [from, to) with val; rest of the array is untouched.
Arrays.copyOf(arr, newLen)	*O(n)*	*O(n)*	Returns a new array of length newLen, copying old values in and padding extra slots with 0/null.
Arrays.copyOfRange(arr, from, to)	*O(n)*	*O(n)*	Returns a new array containing just the sub-range [from, to) of the original.
Arrays.equals(a, b)	*O(n)*	*O(1)*	Checks two 1D arrays have the same length and equal elements at every index (unlike a==b, which just compares references).
Arrays.deepEquals(a, b)	*O(n)*	*O(n)*	Like Arrays.equals but recurses into nested arrays — use for 2D/3D arrays.
Arrays.toString(arr)	*O(n)*	*O(n)*	Builds a readable String like "[1, 2, 3]" for a 1D array — use for printing/debugging.
Arrays.deepToString(arr)	*O(n)*	*O(n)*	Like Arrays.toString but recurses into nested arrays — use for printing 2D/3D arrays.
Arrays.binarySearch(arr, key)	*O(log n)*	*O(1)*	Finds key's index in an ALREADY-SORTED array in O(log n). Returns a negative number if key is absent — sort first or results are undefined.
Arrays.asList(arr)	*O(1)*	*O(1)*	Wraps an Object[] (e.g. Integer[], String[]) as a fixed-size List view backed by the same array — you can set() elements but NOT add()/remove(). Does not work directly on int[].
Arrays.stream(arr).sum()/.max()/.min()	*O(n)*	*O(1)*	Converts an int[] to an IntStream to quickly get its sum, max, or min without writing a loop.

Reverse-sort an int[] (no direct method exists):

```
Integer[] boxed = Arrays.stream(arr).boxed().toArray(Integer[]::new);
Arrays.sort(boxed, Collections.reverseOrder());
```

int[] cannot be sorted in reverse directly — box it to Integer[] first.

2. ArrayList (java.util.ArrayList)

Method	T.C	S.C	Use
add(x)	*O(1)**	*O(1)*	Appends x to the end of the list. O(1) on average (amortized).
add(index, x)	*O(n)*	*O(1)*	Inserts x at position index, shifting later elements right by one. O(n) because of the shift.
get(index)	*O(1)*	*O(1)*	Returns the element at position index. O(1) — ArrayList is backed by an array.
set(index, x)	*O(1)*	*O(1)*	Replaces the element at position index with x and returns the old value.
remove(index)	*O(n)*	*O(1)*	Removes the element at position index (by position!) and shifts later elements left. O(n).
remove(Object o)	*O(n)*	*O(1)*	Removes the first element equal to o (by value!), not by index. Tip: for Integer lists, remove((Integer) 5) removes the value 5, while remove(5) removes the element at index 5.
size()	*O(1)*	*O(1)*	Returns how many elements are currently in the list (note: size(), not length like arrays).
contains(x)	*O(n)*	*O(1)*	Checks whether x is present anywhere in the list by scanning it. O(n).
indexOf(x)	*O(n)*	*O(1)*	Returns the index of the first occurrence of x, or -1 if it isn't in the list.
isEmpty()	*O(1)*	*O(1)*	Returns true if the list has zero elements — shorthand for size() == 0.
clear()	*O(n)*	*O(1)*	Removes every element, leaving the list empty (size() becomes 0).
Collections.sort(list)	*O(n log n)*	*O(n)*	Sorts ascending using natural order. Example: Collections.sort(list) turns [5,2,8] into [2,5,8]. O(n log n).
Collections.sort(list, cmp)	*O(n log n)*	*O(n)*	Sorts using a custom Comparator you provide. Example: Collections.sort(list, (a, b) -> b - a) sorts descending.
Collections.reverse(list)	*O(n)*	*O(1)*	Reverse in place
Collections.max/min(list)	*O(n)*	*O(1)*	Extremes
Collections.frequency(list, x)	*O(n)*	*O(1)*	Count occurrences
list.toArray(new Integer[0])	*O(n)*	*O(n)*	Copies the list's elements into a new Integer[] array of the right size.

3. String & StringBuilder

String is immutable — every modification creates a new object. Use StringBuilder inside loops.

Method	T.C	S.C	Use
s.length()	O(1)	O(1)	Returns the number of characters in the string.
s.charAt(i)	O(1)	O(1)	Returns the single character at index i (0-based). Throws an exception if i is out of range.
s.substring(a,b)	O(n)	O(n)	Returns the characters from index a up to (but not including) index b.
s.indexOf(ch/str)	O(n)	O(1)	Returns the index of the first occurrence of the character/substring, or -1 if not found.
s.lastIndexOf(ch/str)	O(n)	O(1)	Like indexOf but searches from the end, returning the last occurrence's index.
s.split(regex)	O(n)	O(n)	Splits the string on the given regex pattern and returns the pieces as a String[].
s.toCharArray()	O(n)	O(n)	Copies the string's characters into a new char[] so you can index/modify them directly.
s.equals(t) / equalsIgnoreCase(t)	O(n)	O(1)	Compares actual text content (equalsIgnoreCase ignores case). Always use this instead of == for strings, since == compares object references, not content.
s.compareTo(t)	O(n)	O(1)	Compares two strings alphabetically: negative if s < t, 0 if equal, positive if s > t.
s.trim() / s.strip()	O(n)	O(n)	Removes leading/trailing whitespace. strip() (Java 11+) also handles Unicode whitespace that trim() misses.
s.toUpperCase()/.toLowerCase()	O(n)	O(n)	Returns a new string with all letters converted to upper/lower case (original string is unchanged, since Strings are immutable).
s.replace(old,new)	O(n)	O(n)	Returns a new string with every occurrence of old replaced by new.
s.contains(sub)	O(n·m)	O(1)	Returns true if sub appears anywhere inside the string.
s.startsWith()/.endsWith()	O(n)	O(1)	Checks whether the string begins or ends with the given prefix/suffix.
String.valueOf(x)	O(1)	O(1)	Converts almost any type (int, char, boolean, object, etc.) into its String representation.
String.join(",", list)	O(n)	O(n)	Joins a list/array of strings into one string, separated by the given delimiter — e.g. joins ["a","b"] into "a,b".
sb.append(x)	O(1)*	O(1)	Adds x to the end of the StringBuilder's buffer in O(1) amortized time — the efficient alternative to s = s + x.
sb.insert(i,x)	O(n)	O(1)	Inserts x into the buffer at position i, shifting later characters right.

Method	T.C	S.C	Use
sb.deleteCharAt(i)	O(n)	O(1)	Removes the single character at position i from the buffer.
sb.reverse()	O(n)	O(1)	Reverses the buffer's characters in place.
sb.setCharAt(i,ch)	O(1)	O(1)	Overwrites the character at position i with ch.
sb.toString()	O(n)	O(n)	Converts the buffer's contents into an immutable String (call this when you're done building).

> 💡 *Never do s = s + x inside a loop (O(n²) total). Always use StringBuilder.*

4. HashMap / TreeMap / LinkedHashMap

Method	T.C	S.C	Use
map.put(k,v)	O(1)*	O(1)	Inserts key k with value v, or overwrites v if k is already present.
map.get(k)	O(1)*	O(1)	Returns the value mapped to k, or null if k isn't in the map.
map.getOrDefault(k,def)	O(1)*	O(1)	Like get(k), but returns def instead of null when k is missing — avoids a manual null check.
map.containsKey(k)	O(1)*	O(1)	Returns true if the map has an entry for key k.
map.remove(k)	O(1)*	O(1)	Deletes the entry for key k (if present) and returns its old value.
map.putIfAbsent(k,v)	O(1)*	O(1)	Inserts k→v only if k isn't already in the map; otherwise leaves the existing value untouched.
map.merge(k,v,(o,n)->o+n)	O(1)*	O(1)	If k is absent, inserts k→v. If k is present, combines the old value (o) and new value (n) with the given function — the standard idiom for building a frequency map.
map.compute(k,(key,val)->val+1)	O(1)*	O(1)	Recomputes the value for key using a function you supply, based on the current value (val may be null if absent).
map.keySet() / .values() / .entrySet()	O(1)	O(1)	Returns a view for iterating just the keys, just the values, or key-value pairs (Map.Entry) together.
map.size()	O(1)	O(1)	Returns the number of key-value entries in the map.
TreeMap: firstKey()/lastKey()	O(log n)	O(1)	Returns the smallest/largest key currently in the TreeMap (keys are kept sorted automatically).

Method	T.C	S.C	Use
TreeMap: ceilingKey(k)/floorKey(k)	*O(log n)*	*O(1)*	Returns the smallest key ≥ k (ceiling) or largest key ≤ k (floor), or null if none exists.
TreeMap: higherKey(k)/lowerKey(k)	*O(log n)*	*O(1)*	Like ceiling/floor but strict: smallest key > k, or largest key < k.
TreeMap: pollFirstEntry()/pollLastEntry()	*O(log n)*	*O(1)*	Removes and returns the entry with the smallest/largest key.

Frequency counter idiom:

```
map.merge(x, 1, Integer::sum);   // increment count of x
```

5. HashSet / TreeSet

Method	T.C	S.C	Use
set.add(x)	*O(1)**	*O(1)*	Inserts x if it's not already present; does nothing (no error) if it's a duplicate.
set.contains(x)	*O(1)**	*O(1)*	Checks membership in average O(1) time (HashSet) — much faster than scanning a list.
set.remove(x)	*O(1)**	*O(1)*	Removes x from the set if present.
set.retainAll(other)	*O(n)*	*O(1)*	Keeps only the elements also present in other — computes the set intersection, in place.
set.removeAll(other)	*O(n)*	*O(1)*	Removes every element that's also in other — computes the set difference, in place.
set.addAll(other)	*O(m)*	*O(1)*	Adds every element of other into this set — computes the set union, in place.
TreeSet: first()/last()	*O(log n)*	*O(1)*	Returns the smallest/largest element (TreeSet keeps elements sorted automatically).
TreeSet: ceiling(x)/floor(x)	*O(log n)*	*O(1)*	Returns the smallest element ≥ x (ceiling) or largest element ≤ x (floor).
TreeSet: higher(x)/lower(x)	*O(log n)*	*O(1)*	Like ceiling/floor but strict: smallest element > x, or largest element < x.
TreeSet: pollFirst()/pollLast()	*O(log n)*	*O(1)*	Removes and returns the smallest/largest element.

6. Stack (use ArrayDeque, not legacy Stack)

Method	T.C	S.C	Use
stack.push(x)	O(1)*	O(1)	Adds x to the top of the stack (LIFO — last in, first out).
stack.pop()	O(1)	O(1)	Removes and returns the top element. Throws if the stack is empty.
stack.peek()	O(1)	O(1)	Returns the top element without removing it.
stack.isEmpty()	O(1)	O(1)	Returns true if the stack has no elements.

Declaration:

```
Deque<Integer> stack = new ArrayDeque<>();
```

7. Queue / Deque (java.util.ArrayDeque)

Deque can act as Queue, Stack, or double-ended queue. Prefer ArrayDeque over LinkedList.

Method	T.C	S.C	Use
dq.offer(x) / offerLast(x)	O(1)*	O(1)	Adds x to the back of the deque — the standard way to enqueue for FIFO use.
dq.offerFirst(x)	O(1)*	O(1)	Adds x to the front of the deque.
dq.poll() / pollFirst()	O(1)	O(1)	Removes and returns the front element, or null (not an exception) if the deque is empty.
dq.pollLast()	O(1)	O(1)	Removes and returns the back element, or null if empty.
dq.peek() / peekFirst()	O(1)	O(1)	Returns the front element without removing it (null if empty).
dq.peekLast()	O(1)	O(1)	Returns the back element without removing it (null if empty).
dq.isEmpty() / size()	O(1)	O(1)	Checks whether the deque is empty, or returns how many elements it holds.

> 💡 *Sliding window maximum: keep a Deque<Integer> of indices, values monotonic decreasing.*

8. PriorityQueue (Min-Heap by default)

Method	T.C	S.C	Use
new PriorityQueue<>()	O(1)	O(1)	Creates a min-heap: poll() always returns the smallest element first, using natural ordering.
new PriorityQueue<>(Collections.reverseOrder())	O(1)	O(1)	Creates a max-heap: poll() always returns the largest element first.
new PriorityQueue<>((a,b)->a[0]-b[0])	O(1)	O(1)	Creates a heap with a custom ordering rule — here, e.g. int[] pairs are ordered by their first element ascending.
pq.offer(x) / add(x)	O(log n)	O(1)	Inserts x into the heap, maintaining heap order. O(log n).
pq.poll()	O(log n)	O(1)	Removes and returns the top (min or max, depending on setup) element. O(log n).
pq.peek()	O(1)	O(1)	Returns the top element without removing it. O(1).
pq.isEmpty() / size()	O(1)	O(1)	Checks whether the heap is empty, or returns how many elements it holds.

> 💡 *Iterating a PriorityQueue does NOT give sorted order — only repeated poll() does.*

9. Math & Integer/Long utilities

Method	T.C	S.C	Use
Math.max(a,b) / min(a,b)	O(1)	O(1)	Returns the larger/smaller of two values.
Math.abs(x)	O(1)	O(1)	Returns the absolute (non-negative) value of x.
Math.pow(base,exp)	O(1)	O(1)	Returns base raised to the power exp, as a double (not an int).
Math.sqrt(x)	O(1)	O(1)	Returns the square root of x, as a double.
Math.ceil(x) / floor(x)	O(1)	O(1)	Rounds x up (ceil) or down (floor) to the nearest whole number, returned as a double.
Math.round(x)	O(1)	O(1)	Rounds x to the nearest whole number (0.5 rounds up).

Method	T.C	S.C	Use
Math.log(x) / log10(x)	O(1)	O(1)	Returns the natural logarithm (base e) or base-10 logarithm of x.
Integer.MAX_VALUE / MIN_VALUE	O(1)	O(1)	The largest/smallest value an int can hold: 2147483647 / -2147483648. Adding 1 past MAX_VALUE silently wraps to MIN_VALUE (overflow).
Long.MAX_VALUE / MIN_VALUE	O(1)	O(1)	Like Integer's constants but for long, which has a much larger range — use when sums/products might exceed ~2.1 billion.
Integer.parseInt(str)	O(n)	O(1)	Converts a numeric String into an int. Throws NumberFormatException if str isn't a valid integer.
Integer.toBinaryString(x)	O(1)	O(1)	Returns x's binary representation as a String (no leading zeros), e.g. 5 → "101".
Integer.bitCount(x)	O(1)	O(1)	Returns how many bits are set to 1 in x (a.k.a. popcount).
Integer.numberOfTrailingZeros(x)	O(1)	O(1)	Returns the position (0-indexed from the right) of the lowest set bit.
Integer.compare(a,b)	O(1)	O(1)	Returns -1, 0, or 1 depending on whether a < b, a == b, or a > b — safer than a-b, which can overflow for extreme values.

10. Character utilities

Method	T.C	S.C	Use
Character.isDigit(ch)	O(1)	O(1)	Returns true if ch is a digit character ('0' through '9').
Character.isLetter(ch)	O(1)	O(1)	Returns true if ch is an alphabetic letter.
Character.isLetterOrDigit(ch)	O(1)	O(1)	Returns true if ch is either a letter or a digit.
Character.isUpperCase/isLowerCase(ch)	O(1)	O(1)	Checks whether ch is uppercase or lowercase.
Character.toUpperCase/toLowerCase(ch)	O(1)	O(1)	Returns the upper/lower-case version of ch.
Character.getNumericValue(ch)	O(1)	O(1)	Converts a digit character to its int value, e.g. '7' → 7 (useful when parsing digits from a string one char at a time).

11. Collections utility class

Method	T.C	S.C	Use
Collections.sort(list)	*O(n log n)*	*O(n)*	Sort ascending
Collections.reverse(list)	*O(n)*	*O(1)*	Reverse order
Collections.shuffle(list)	*O(n)*	*O(1)*	Randomly reorders the list's elements in place (like shuffling a deck of cards).
Collections.max/min(list)	*O(n)*	*O(1)*	Extremes
Collections.swap(list,i,j)	*O(1)*	*O(1)*	Swaps the elements at positions i and j in place.
Collections.unmodifiableList(list)	*O(1)*	*O(1)*	Returns a read-only wrapper around list — attempts to modify it throw UnsupportedOperationException. The underlying list can still change if modified through the original reference.
Collections.nCopies(n,x)	*O(1)*	*O(1)*	Returns an immutable list containing x repeated n times, e.g. nCopies(3, 0) → [0, 0, 0].

12. Fast I/O (critical for timed judges like TCS iON)

BufferedReader + StringTokenizer (fast, preferred for large input):

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
int n = Integer.parseInt(br.readLine().trim());
StringTokenizer st = new StringTokenizer(br.readLine());
int a = Integer.parseInt(st.nextToken());
int b = Integer.parseInt(st.nextToken());
```

> 💡 *Scanner is simpler but noticeably slower on large inputs — prefer BufferedReader under strict time limits.*

GCD / LCM (write manually):

```
static int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
static int lcm(int a, int b) { return a / gcd(a, b) * b; }
```

> 💡 *Use long for sums/products that could exceed ~2.1 billion — int overflows silently.*

13. Bit Manipulation (operators, not methods)

Method	T.C	S.C	Use
x & 1	*O(1)*	*O(1)*	Checks the lowest bit: result is 1 if x is odd, 0 if x is even.
x >> 1	*O(1)*	*O(1)*	Shifts bits right by one position — equivalent to integer division by 2.
x << 1	*O(1)*	*O(1)*	Shifts bits left by one position — equivalent to multiplying by 2.
x >> n	*O(1)*	*O(1)*	Arithmetic (signed) right shift: **divides by 2^n**, rounding toward -infinity. Vacated high bits are filled with the SIGN bit, so a negative x stays negative.
x >>> n	*O(1)*	*O(1)*	Logical (unsigned) right shift: vacated high bits are always filled with 0, regardless of sign. Use when treating x as an unsigned 32-bit value, e.g. (x >>> 1) for an overflow-safe midpoint average.
x << n	*O(1)*	*O(1)*	Left shift by n **multiplies by 2^n**. Bits shifted past bit 31 (int) or bit 63 (long) are silently discarded — watch for overflow, e.g. 1 << 31 overflows int to Integer.MIN_VALUE.
x & ~(1 << i)	*O(1)*	*O(1)*	Clears bit i to 0, leaving all other bits unchanged.
x ^ (1 << i)	*O(1)*	*O(1)*	Flips (toggles) bit i: 0 becomes 1, 1 becomes 0.
(x >> i) & 1	*O(1)*	*O(1)*	Checks whether bit i is set: returns 1 if set, 0 if not.
x & -x	*O(1)*	*O(1)*	Isolates the lowest set bit of x, zeroing out all other bits (relies on two's-complement representation).
x & y	*O(1)*	*O(1)*	AND — result bit is 1 only when both bits are 1. Used to mask/isolate bits, e.g. x & 0xFF keeps only the lowest byte.
x | y	*O(1)*	*O(1)*	OR — result bit is 1 if either bit is 1. Used to set/combine bits, e.g. x | (1 << i) turns bit i on.
x ^ y	*O(1)*	*O(1)*	XOR — result bit is 1 when the two input bits differ, 0 when they match. Toggles bits, swaps two values without a temp variable, and finds the single non-duplicate element in an array (XOR everything together).
~x	*O(1)*	*O(1)*	NOT (bitwise complement) — flips every bit. In two's complement this equals -(x+1), so ~5 == -6 and ~0 == -1.
~(x & y)	*O(1)*	*O(1)*	NAND — Java has no direct operator; write it as ~(x & y). Result bit is 1 unless both input bits are 1.
~(x | y)	*O(1)*	*O(1)*	NOR — Java has no direct operator; write it as ~(x | y). Result bit is 1 only when both input bits are 0.

Method	T.C	S.C	Use
~(x ^ y)	O(1)	O(1)	XNOR — Java has no direct operator; write it as ~(x ^ y). Result bit is 1 when the two input bits are the same.

14. Number Base Conversions

Java built-ins for converting between decimal, binary, octal, and hexadecimal. Integer handles up to 32-bit values; use Long equivalents for larger numbers.

Method	T.C	S.C	Use
Integer.toBinaryString(n)	O(1)	O(1)	Decimal int to binary String. 10 → "1010". No leading zeros.
Integer.toOctalString(n)	O(1)	O(1)	Decimal int to octal String. 8 → "10".
Integer.toHexString(n)	O(1)	O(1)	Decimal int to hex String (lowercase). 255 → "ff".
Integer.parseInt(s, 2)	O(n)	O(1)	Binary String to decimal int. "1010" → 10.
Integer.parseInt(s, 8)	O(n)	O(1)	Octal String to decimal int. "17" → 15.
Integer.parseInt(s, 16)	O(n)	O(1)	Hex String to decimal int. "ff" → 255.
Integer.toString(n, radix)	O(1)	O(1)	Decimal to any base (2–36) as String. Integer.toString(255, 16) → "ff".
Long.toBinaryString(n)	O(1)	O(1)	Same as Integer version but accepts long values (64-bit).
Long.parseLong(s, 2)	O(n)	O(1)	Binary String to long. Handles values > Integer.MAX_VALUE.
Integer.bitCount(n)	O(1)	O(1)	Number of 1-bits in binary representation (popcount). 7 → 3.
Integer.highestOneBit(n)	O(1)	O(1)	Returns value with only the highest set bit kept. 10 → 8.
Integer.reverse(n)	O(1)	O(1)	Reverses all 32 bits of n.
Integer.reverseBytes(n)	O(1)	O(1)	Reverses the 4 bytes of n (byte-order swap).

Binary ↔ Decimal examples:

```
int dec = 42;
String bin = Integer.toBinaryString(dec);   // "101010"
int back = Integer.parseInt(bin, 2);        // 42

// With leading zeros (padded to 8 bits)
String padded = String.format("%8s", bin).replace(' ', '0'); // "00101010"
```

All-bases conversion snippet:

```
int n = 255;
System.out.println(Integer.toBinaryString(n));  // 11111111
System.out.println(Integer.toOctalString(n));   // 377
System.out.println(Integer.toHexString(n));     // ff
System.out.println(Integer.toString(n, 36));    // 73  (base-36)
```

Parse any base back to decimal:

```
int fromBin = Integer.parseInt("11111111", 2);   // 255
int fromOct = Integer.parseInt("377",      8);  // 255
int fromHex = Integer.parseInt("ff",      16);  // 255
```

💡 Integer.parseInt(s, radix) throws NumberFormatException if the string contains digits invalid for that base — wrap in try-catch when parsing user input.
💡 Long.parseLong(s, 2) / Long.toBinaryString(n) mirror the Integer versions but accept 64-bit values — use when bit-mask values exceed ~2.1 billion.

15. Quick decision table — which structure to use

Method	Use
Fast lookup by key	HashMap — O(1) average get/put by key, no ordering guarantee.
Sorted keys, range queries	TreeMap — keeps keys sorted, supports floor/ceiling/range operations in O(log n).
Unique elements, fast membership	HashSet — O(1) average contains(), no ordering guarantee.
Unique + sorted	TreeSet — like HashSet but keeps elements sorted, O(log n) operations.
LIFO (undo, parentheses, DFS)	Deque used as a Stack via push()/pop()/peek() — last in, first out.
FIFO (BFS, scheduling)	Deque used as a Queue via offer()/poll()/peek() — first in, first out.
Repeatedly get min/max	PriorityQueue — O(log n) insert/remove while always giving you the current min or max.
Index-based fast access	ArrayList / array — O(1) get(index)/set(index) by position.
Insert/delete at both ends	ArrayDeque — O(1) add/remove at both the front and back.